



MINI RAPPORT TECHNIQUE

Mécanismes internes du Pipeline RAG

Vue purement informatique — concurrence, traitement d'image,
algorithmes, réseau et indexation

Auteur :
Équipe RAG / Projet Waraq

Date de publication :
10 août 2026

Table des matières

1. Concurrence : comment le RAG s'exécute sans bloquer le reste de l'application	1
2. Extraction de texte natif (PyMuPDF)	1
3. Détection « page scannée » — heuristique statistique, pas de ML . . .	1
4. OCR — RapidOCR sur ONNX Runtime	2
5. Détection du type de document — pattern matching pur	2
6. Extraction de sous-documents — manipulation structurelle de PDF .	3
7. Conversion DOC → PDF — appel de processus externe	3
8. Parsing Excel	4
9. Génération de PDF (BDP) — moteur de mise en page déclaratif . . .	4
10.Appels réseau vers Ollama (modèles locaux)	4
11.Chunking — algorithme de fenêtre glissante	5
12.Embeddings — vectorisation	5
13.Indexation vectorielle — ChromaDB / HNSW	5
14.Recherche sémantique — mécanique de la requête	6
15.Persistance et checkpointing	6
Résumé de la mécanique globale	6

1. Concurrency : comment le RAG s'exécute sans bloquer le reste de l'application

Le fichier `rag_trigger.py` lance un `threading.Thread(daemon=True)`. Concrètement :

- Un thread est un fil d'exécution du **même processus Python**, mais avec sa propre pile d'appels. `daemon=True` signifie que ce thread ne retient pas le processus en vie : si l'application s'arrête, le thread est tué sans attendre.
- Dans ce thread, une **nouvelle session SQLAlchemy** (`SessionLocal()`) est créée. Une session ORM n'est pas thread-safe : chaque thread doit avoir la sienne, sinon on obtient des corruptions d'état ou des erreurs de connexion partagée sur le pool.
- À l'intérieur du thread, le code appelle `asyncio.run(rag_pipeline_service.execute_rag_pipeline(...))`. C'est une subtilité importante : `asyncio.run()` **crée une nouvelle boucle d'événements** (event loop) à chaque appel. On ne peut pas appeler `asyncio.run()` depuis une coroutine déjà en cours d'exécution dans la boucle principale de FastAPI (cela lèverait `RuntimeError: asyncio.run() cannot be called from a running event loop`). C'est justement pour cette raison que tout ce bloc tourne dans un **thread OS classique**, sans boucle `asyncio` préexistante : cela permet de mélanger du code ORM synchrone (SQLAlchemy) avec des appels HTTP asynchrones (`httpx.AsyncClient`) sans conflit.
- Les documents sont traités **séquentiellement** (boucle `for`), pas en parallèle. C'est voulu : chaque étape du pipeline charge un modèle local (Ollama) en RAM ; paralléliser ferait tourner plusieurs modèles simultanément et ferait exploser la consommation mémoire sur une machine CPU-only à RAM limitée.

2. Extraction de texte natif (PyMuPDF)

`fitz.open(file_path)` parse la table xref du PDF (structure interne indexant tous les objets : pages, polices, flux de contenu).

`page.get_text("text", sort=True)` :

- lit le flux de contenu de la page (opérateurs de dessin de texte avec leurs coordonnées `x/y`) ;
- `sort=True` réordonne les blocs de texte par position (haut → bas, gauche → droite) plutôt que dans l'ordre brut du flux PDF, ce qui évite un texte « mélangé » sur des mises en page à plusieurs colonnes ou avec des tableaux ;
- cela ne fonctionne **que** si le PDF possède une couche texte native (police vectorielle intégrée). Sur un scan (image pure), l'appel renvoie une chaîne vide ou quasi vide.

3. Détection « page scannée » — heuristique statistique, pas de ML

`_ocr_est_de_mauvaise_qualite(texte)` applique trois tests successifs sur la chaîne extraite :

```

1. len(texte) < 250 -> rejete (trop court)
2. nb_lettres / len(texte) < 0.60 -> rejete (trop peu de lettres)
   ou nb_lettres = re.findall(r"[A-Za-z\u0600-\u06FF]", texte)
   (plage Unicode Latin étendu + bloc arabe U+0600 à U+06FF)
3. count(r"[*#=<>|~]") >= 2 -> rejete (caractères "parasites")

```

Aucun modèle n'est impliqué ici : c'est un simple filtre à base de comptage de caractères et de recherche par expressions régulières. Le seuil de 250 caractères et le ratio de 0.60 sont des constantes empiriques codées en dur.

4. OCR — RapidOCR sur ONNX Runtime

ONNX (Open Neural Network Exchange) est un format de modèle portable : les réseaux de neurones entraînés (souvent avec PyTorch) sont exportés en `.onnx` et exécutés via un runtime d'inférence dédié (`onnxruntime`), sans dépendre du framework d'entraînement. Avantage direct ici : inférence CPU efficace, pas besoin de GPU ni de PyTorch au runtime — cohérent avec la contrainte « CPU-only » du serveur.

RapidOCR enchaîne en interne **trois modèles ONNX** :

1. **Détection** : localise les zones de texte (bounding boxes) dans l'image.
2. **Classification d'orientation** : corrige les boîtes tournées à 180°.
3. **Reconnaissance** (type CRNN) : convertit chaque zone image en chaîne de caractères accompagnée d'un score de confiance.

Pipeline concret côté code (`extraire_page_complete_fast_ocr`) :

```

page.get_pixmap(matrix=fitz.Matrix(2.0, 2.0)) # rasterise à 200% de résolution
np.frombuffer(pix.samples).reshape((H, W, 3)) # bytes bruts -> tableau numpy (
    RGB)
cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) # conversion en niveaux de gris
engine(gray) # inference RapidOCR

```

Le zoom x2 avant OCR est important : une police à 10pt rendue à 72 DPI standard devient trop petite/floue pour le modèle de reconnaissance ; doubler la résolution améliore nettement le taux de reconnaissance.

Le résultat de `engine(image)` est une liste de tuples (`coordonnees_boite`, `texte`, `confiance`). Le code ne garde que les lignes avec `confiance >= 0.40` — un seuil assez permissif (favorise le rappel au détriment de la précision : on préfère garder du texte bruité plutôt que d'en perdre).

5. Détection du type de document — pattern matching pur

`detecter_types_documents()` procède en trois étapes :

1. normalisation du texte : passage en minuscules, unification des apostrophes typographiques et des accents graves vers l'apostrophe simple, compression des espaces multiples via `re.sub(r"s+", " ", text)` ;

2. pour chaque type dans `DOCUMENT_PATTERNS` (dictionnaire `{type: [regex, ...]}`), application de `re.search(pattern, texte, re.IGNORECASE)` ;
3. dès qu'une regex du type considéré trouve une correspondance, le type est ajouté à la liste des résultats et l'on passe au type suivant (`break` sur la boucle interne, pas sur la boucle externe) — un même texte peut donc matcher **plusieurs types simultanément** (par exemple un CPS peut contenir à la fois le motif « CPS » et un motif « bordereau des prix » sur une page annexe).

Aucune notion sémantique n'intervient ici : il s'agit d'un matching de chaînes déterministe, rapide (de l'ordre de quelques microsecondes par page), utilisé comme filtre avant d'invoquer des modèles plus coûteux.

6. Extraction de sous-documents — manipulation structurelle de PDF

`extraire_pages_modeles_pdf()` scanne le PDF page par page, applique la détection de type sur le texte de chaque page, puis regroupe les indices de pages par type détecté (avec une marge de contexte de ± 1 page pour ne pas couper un modèle à cheval sur deux pages).

Pour chaque type, un nouveau PDF est construit :

```
nouveau_pdf = fitz.open() # PDF vide en memoire
nouveau_pdf.insert_pdf(doc, from_page=i, to_page=i) # copie structurelle de la
page
```

`insert_pdf()` copie les **objets PDF** (police, texte vectoriel, images embarquées) tels quels depuis le document source — ce n'est pas un rendu ou une re-rasterisation, donc aucune perte de qualité, et le texte reste sélectionnable s'il l'était dans l'original.

7. Conversion DOC → PDF — appel de processus externe

```
subprocess.run(
    ["libreoffice", "--headless", "--convert-to", "pdf",
     "--outdir", output_dir, file_path],
    timeout=120
)
```

- `--headless` : LibreOffice démarre sans interface graphique, en tant que processus séparé du processus Python ;
- `subprocess.run()` est **bloquant** : le thread Python attend la fin du processus (ou le dépassement du délai de 120 secondes) avant de continuer ;
- le fichier de sortie est retrouvé par convention de nommage (`{nom_du_fichier}.pdf` dans le dossier temporaire), et non via une valeur de retour explicite de LibreOffice.

8. Parsing Excel

```
pd.read_excel(file_path, sheet_name=None, header=None,
              engine=engine, dtype=object)
```

- `sheet_name=None` : lit **toutes** les feuilles d'un coup, renvoie un dict `{nom_feuille: DataFrame}` ;
- le paramètre `engine` vaut `xlrd` pour le format binaire legacy `.xls`, et `openpyxl` pour `.xlsx/.xlsm` (qui sont en réalité des archives ZIP contenant du XML) ;
- `header=None, dtype=object` empêche pandas de traiter la première ligne comme un en-tête de colonnes et d'inférer des types (`int`, `float`, `date`) — les valeurs brutes de chaque cellule sont conservées pour ne rien perdre à l'affichage ;
- les cellules vides deviennent `NaN` (de type `float`) par défaut, d'où le test explicite `pd.isna(value)` avant conversion en chaîne.

9. Génération de PDF (BDP) — moteur de mise en page déclaratif

ReportLab (`SimpleDocTemplate`) fonctionne par accumulation d'objets « flowables » (`Paragraph`, `Table`, `Spacer`, `PageBreak`) dans une liste (`story`). L'appel `doc.build(story)` calcule ensuite automatiquement la pagination, les sauts de page et le placement — le développeur ne gère jamais les coordonnées x/y manuellement, contrairement à un dessin PDF bas niveau.

10. Appels réseau vers Ollama (modèles locaux)

Tous les appels aux modèles (GLM-OCR, Granite, BGE-M3) passent par des requêtes HTTP REST vers un serveur Ollama local (`settings.OLLAMA_BASE_URL`), via `httpx.AsyncClient` (client HTTP non bloquant, compatible avec `asyncio`).

Points techniques clés :

- `"stream": false` : demande la réponse complète en un seul bloc JSON plutôt qu'un flux token par token ;
- `"keep_alive": settings.OLLAMA_KEEP_ALIVE` : indique à Ollama combien de temps garder le modèle chargé en mémoire après la requête. À 0, le modèle est déchargé immédiatement — nécessaire pour enchaîner GLM-OCR puis Granite sur une machine à RAM limitée sans les avoir tous les deux en mémoire simultanément ;
- `"format": "json"` (utilisé uniquement pour Granite) : contraint le décodeur du modèle à ne produire que des tokens formant un JSON syntaxiquement valide (décodage contraint par grammaire), une garantie plus forte qu'une simple instruction dans le prompt ;
- le champ `response.json().get("response")` est une **chaîne de caractères**, qui contient elle-même du JSON — d'où le double parsing : une fois pour la réponse HTTP, une fois via `json.loads()` sur le contenu de `"response"`.

11. Chunking — algorithme de fenêtre glissante

```
words = text.split() # tokenisation naive par espaces
step = chunk_size - overlap # 800 - 150 = 650

for start in range(0, len(words), step):
    end = start + chunk_size
    chunk = words[start:end]
    if end >= len(words):
        break
```

Exemple concret sur un texte de 2000 mots (`chunk_size=800`, `overlap=150`, `step=650`) :

Chunk	start	end	Mots couverts
0	0	800	0–800
1	650	1450	650–1450
2	1300	2100 → 2000	1300–2000 (dernier, <code>break</code>)

La zone de recouvrement (150 mots partagés entre deux chunks consécutifs) évite qu’une information charnière (par exemple une phrase coupée entre deux blocs) ne soit totalement perdue pour la recherche vectorielle. Le découpage se fait sur les espaces (`str.split()`), donc indépendamment de l’alphabet — valable aussi bien en français qu’en arabe, contrairement à un découpage par caractères qui poserait problème sur l’arabe (script cursif, formes contextuelles des lettres).

12. Embeddings — vectorisation

L’appel `POST /api/embed` avec `{"model": "bge-m3", "input": [chunk1, chunk2, ...]}` renvoie une **liste de vecteurs denses** (un par chunk), typiquement de dimension 1024 pour BGE-M3. Chaque vecteur est la sortie d’un encodeur Transformer, obtenu en agrégeant (pooling) les représentations token par token du texte en un seul vecteur de taille fixe. Ce vecteur capture le « sens » du texte dans un espace continu : deux textes proches sémantiquement produisent des vecteurs proches géométriquement, même s’ils ne partagent aucun mot en commun.

13. Indexation vectorielle — ChromaDB / HNSW

`chromadb.PersistentClient(path=...)` stocke tout sur disque : métadonnées en SQLite, et un **index HNSW** (Hierarchical Navigable Small World) pour les vecteurs.

- HNSW est une structure de graphe multi-couches permettant une recherche du plus proche voisin **approximative** en temps quasi logarithmique, au lieu d’une comparaison brute contre tous les vecteurs ($O(n)$). C’est ce qui permet à ChromaDB de rester rapide même avec des milliers de chunks indexés.
- `metadata={"hnsw:space": "cosine"}` configure la métrique de distance utilisée pour construire le graphe : la **similarité cosinus** (angle entre deux vecteurs), standard pour les embeddings texte, où la direction du vecteur porte plus d’information que sa norme.

- `collection.upsert(ids=[...], ...)` : les identifiants sont construits de façon déterministe (`doc_{document_id}_chunk_{i}`). Réinsérer avec le même identifiant **remplace** l'entrée existante (vecteur, texte et métadonnées) au lieu de la dupliquer — propriété d'idempotence utile en cas de retraitement du même document.

14. Recherche sémantique — mécanique de la requête

```
query_emb = embed(query_prompt)
collection.query(
    query_embeddings=[query_emb],
    n_results=6,
    where=filtre_optionnel
)
```

1. Le texte de requête est transformé en vecteur avec le **même modèle** que celui utilisé pour indexer les chunks (obligatoire : les vecteurs doivent vivre dans le même espace).
2. ChromaDB parcourt le graphe HNSW pour retrouver les k vecteurs les plus proches en distance cosinus.
3. Le paramètre **where** applique en plus un filtre exact sur les métadonnées (par exemple `document_id`) — combinaison d'un filtre structuré classique et d'une recherche approximative par similarité.
4. Résultat : les chunks sont retrouvés par **proximité de sens**, et non par correspondance de mots-clés — une requête « délai de dépôt des offres » peut ainsi remonter un chunk contenant « date limite de remise des plis » sans partager un seul mot exact.

15. Persistance et checkpointing

- `RAGAnalysisResult.rag_analysis` est stocké en colonne **JSONB** (PostgreSQL) : contrairement à un simple champ texte, le JSON est stocké dans un format binaire indexable, interrogeable directement en SQL (par exemple `WHERE rag_analysis->>'objet' = ...`) sans devoir tout reparser côté application.
- Le code effectue un **commit intermédiaire** juste après l'extraction de texte, avant le chunking : `rag_entry.extracted_text = enhanced_text; db.commit()`. Il s'agit d'une stratégie de checkpointing — si une étape ultérieure (embeddings, LLM) échoue, le texte déjà extrait n'est pas perdu, seule l'analyse reste manquante.

Résumé de la mécanique globale

Le pipeline est en réalité une chaîne d'appels **majoritairement stateless** (chaque étape reçoit une entrée, produit une sortie, sans état partagé caché) orchestrés par du code Python synchrone/asynchrone classique :

```
Thread OS (isolation)
-> extraction fichier (parsing PDF/DOCX/XLS + OCR ONNX si nécessaire)
-> filtrage regex (detection de type)
-> HTTP POST Ollama (reformattage texte)
```



```
-> decoupage en fenetres glissantes (liste de strings)
-> HTTP POST Ollama (vecteurs denses)
-> ecriture dans un index HNSW persistant (ChromaDB)
-> HTTP POST Ollama (requete vectorielle -> JSON contraint)
-> ecriture SQL (checkpoint + resultat final)
```

Chaque brique (OCR, embeddings, LLM) est interchangeable indépendamment des autres — c'est ce découplage technique, plus que la logique métier elle-même, qui constitue la partie la plus intéressante de l'architecture.